

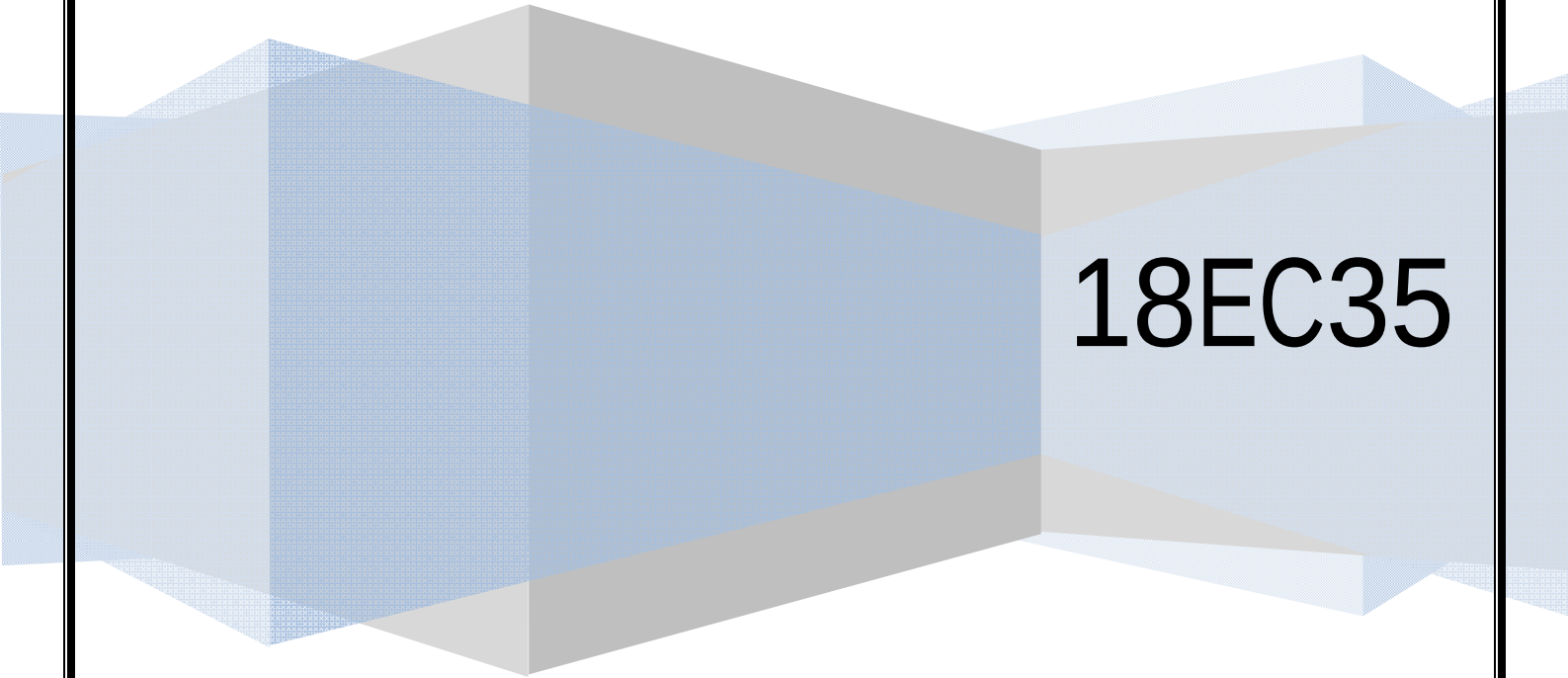
MODULE 3

INPUT/OUTPUT

ORGANIZATION

COMPUTER ORGANISATION AND ARCHITECTURE

PREETHI T S, Assistant Professor, ECE Dept., SCE



18EC35

MODULE 3

CONTENTS

1 INPUT/OUTPUT ORGANIZATION

1.1 Accessing I/O Devices

1.2 Interrupts

1.2.1 Interrupt Hardware

1.2.2 Enabling and Disabling Interrupts

1.2.3 Handling Multiple Devices

1.2.4 Controlling Device Requests

1.3 Direct Memory Access

2 QUESTION BANK

3 EXERCISE PROBLEMS AND SOLUTIONS

SYLLABUS

Input/Output Organization

-  Accessing I/O Devices,
-  Interrupts - Interrupt Hardware
-  Enabling and Disabling Interrupts
-  Handling Multiple Devices
-  Controlling Device Requests
-  Direct Memory Access (upto 4.2.4 and 4.4 except 4.4.1 of Chap 4 of text).

PART 1**INPUT/OUTPUT ORGANIZATION**

In this module we will consider the input/output (I/O) capability of computers as seen from the programmer's point of view. We will present only basic I/O operations, which are provided in all computers. This knowledge will enable to perform interesting and useful exercises on equipment found in a typical teaching laboratory environment.

1.1 ACCESSING I/O DEVICES

- A simple arrangement to connect I/O devices to a computer is to use a single bus arrangement, as shown in Figure 1. 1.
- The bus enables all the devices connected to it to exchange information.
- Typically, it consists of three sets of lines used to carry address, data, and control signals.
- Each I/O device is assigned a unique set of addresses. When the processor places a particular address on the address lines, the device that recognizes this address responds to the commands issued on the control lines.
- The processor requests either a read or a write operation, and the requested data are transferred over the data lines.
- When I/O devices and the memory share the same address space, the arrangement is called *memory-mapped I/O*.

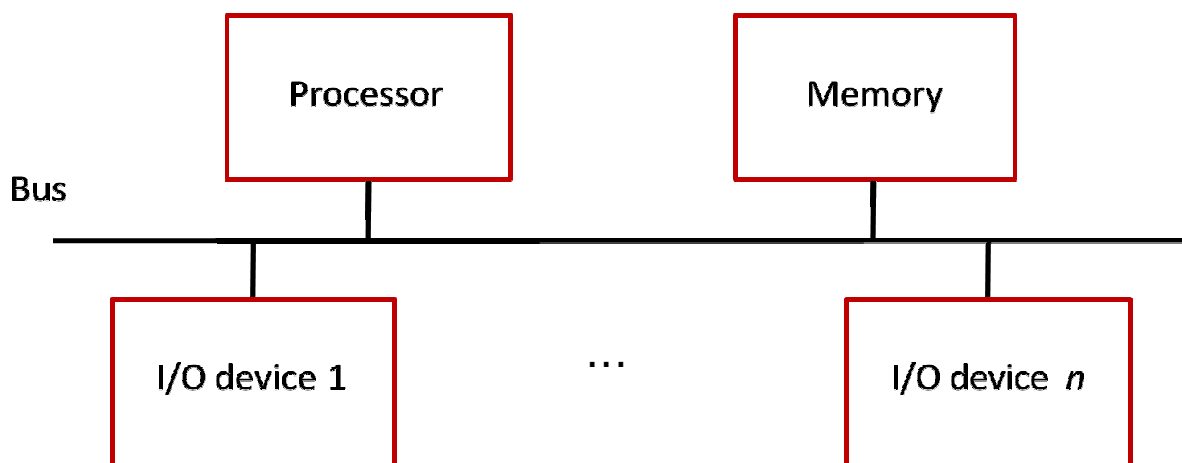


Figure 1.1 A single bus structure.

- With memory-mapped I/O, any machine instruction that can access memory can be used to transfer data to or from an I/O device.
- ***For example***, if **DATAIN** is the address of the input buffer associated with the keyboard, the instruction

Move DATAIN, R0

reads the data from DATAIN and stores them into processor register R0.

- Similarly, the instruction

Move R0, DATAOUT

sends the contents of register R0 to location DATAOUT, which may be the output data buffer of a display unit or a printer.

- Most computer systems use **memory-mapped I/O**. Some processors have special **In** and **Out** instruction to perform I/O transfers.
- When building a computer system, the designer has the option of connecting I/O devices to use the special I/O address space or simply incorporating them as part of the memory address space.
- The latter approach is by far the most common as it leads to simpler software.
- One advantage of a separate I/O address space is that I/O devices deal with fewer address lines.
- Note that a separate I/O address space does not necessarily mean that the I/O address lines are physically separate from the memory address lines.
- A special signal on the bus indicates that the requested read or write transfer is an I/O operation. When this signal is asserted, the memory unit ignores the requested transfer.
- The I/O devices examine the low-order bits of the address bus to determine whether they should respond.
- Figure 1.2 illustrates the hardware required to connect an I/O device to the bus.
- The address decoder enables the device to recognize its address when this address appears on the address lines.
- The data register holds the data being transferred to or from the processor.
- The status register contains information relevant to the operation of the I/O device.
- Both the data and status registers are connected to the data bus and assigned unique addresses.
- The address decoder, the data and status registers, and the control circuitry required to coordinate I/O transfers constitute the device's **interface circuit**.
- I/O devices operate at speeds that are vastly different from that of the processor.

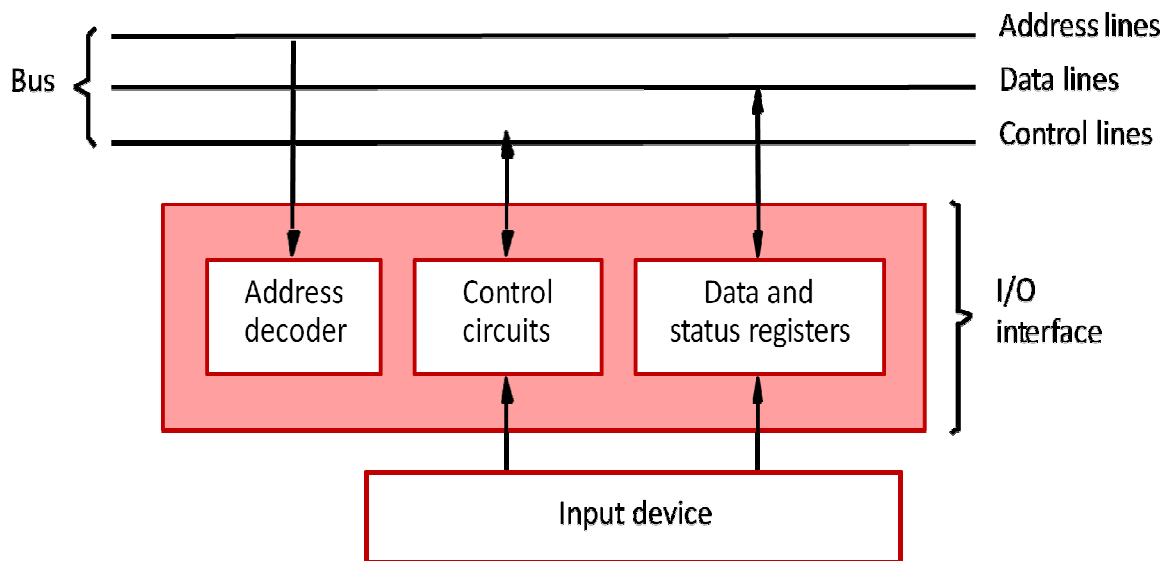


Figure 1.2 I/O interface for an input device.

- When a human operator is entering characters at a keyboard, the processor is capable of executing millions of instructions between successive character entries.
- An instruction that reads a character from the keyboard should be executed only when a character is available in the input buffer of the keyboard interface.
- Also, we must make sure that an input character is read only once.

Program-Controlled I/O

- For an input device such as a keyboard, a status flag, SIN, is included in the interface circuit as part of the status register.
- This flag is set to 1 when a character is entered at the keyboard and cleared to 0 once this character is read by the processor.
- Hence, by checking the SIN flag, the software can ensure that it is always reading valid data,
- This is often accomplished in a program loop that repeatedly reads the status register and checks the state of SIN.
- When SIN becomes equal to 1, the program reads the input data register.
- A similar procedure can be used to control output operations using an output status flag, SOUT.

Example for Program-Controlled I/O

- To review the basic concepts, let us consider a simple example of I/O operations involving a keyboard and a display device in a computer system.
- The four registers shown in Figure 1.3 are used in the data transfer operations.
- Register **STATUS** contains two control flags, **SIN** and **SOUT**, which provide status information for the keyboard and the display unit, respectively.
- The two flags **KIRQ** and **DIRQ** in this register are used in conjunction with interrupts.
- Data from the keyboard are made available in the **DATAIN** register, and data sent to the display are stored in the **DATAOUT** register.

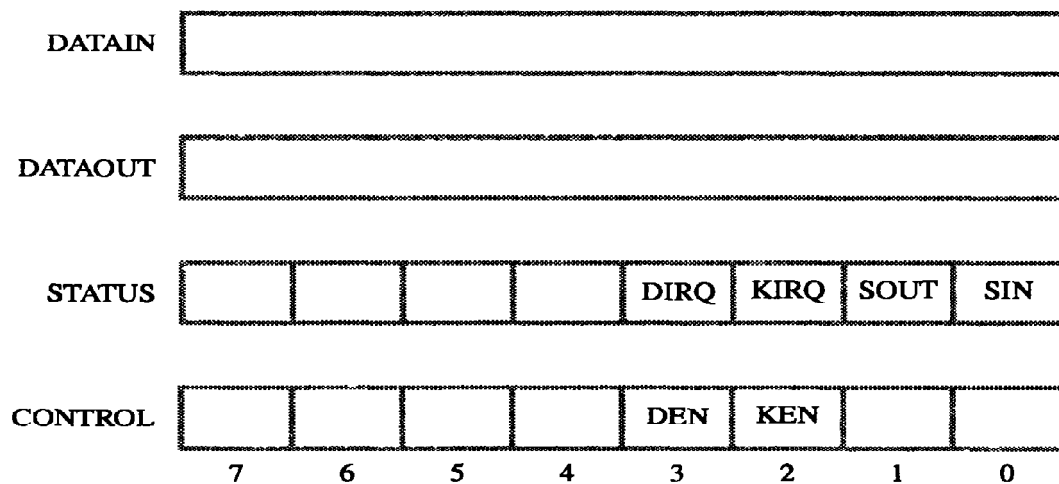


Figure 1.3 Registers in keyboard and display interfaces.

	Move	#LINE,R0	Initialize memory pointer.
WAITK	Test Bit	#0,STATUS	Test SIN.
	Branch=0	WAITK	Wait for character to be entered.
	Move	DATAIN,R1	Read character.
WAITD	TestBit	#1,STATUS	Test SOUT.
	Branch=0	WAITD	Wait for display to become ready.
	Move	R1,DATAOUT	Send character to display.
	Move	R1,(R0)+	Store character and advance pointer.
	Compare	#80D,R1	Check if Carriage Return.
	Branch ≠ 0	WAITK	If not, get another character.
	Move	#\$0A,DATAOUT	Otherwise, send Line Feed.
	Call	PROCESS	Call a subroutine to process the input line

Figure 1.4 A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display

- The program in Figure 1. 4 reads a line of characters from the keyboard and stores it in a memory buffer starting at location **LINE**.
- Then, it calls a subroutine **PROCESS** to process the input line. As each character is read, it is *echoed back* to the display.
- Register R0 is used as a pointer to the memory buffer area. The contents of R0 are updated using the Autoincrement addressing mode so that successive characters are stored in successive memory locations.
- Each character is checked to see if it is the **Carriage Return (CR)** character, which has the ASCII code **OD** (hex).
- If it is, a **Line Feed** character (ASCII code OA) is sent to move the cursor one line down on the display and subroutine **PROCESS** is called.
- Otherwise, the program loops back to wait for another character from the keyboard.
- This example illustrates program-controlled I/O, in which the processor repeatedly checks a status flag to achieve the required synchronization between the processor and an input or output device.

There are two other commonly used mechanisms for implementing I/O operations:

1. Interrupts

2. Direct memory access.

In the case of **interrupts**, synchronization is achieved by having the I/O device send a special signal over the bus whenever it is ready for data transfer operation.

Direct memory access is a technique used for high speed I/O devices. It involves having the device interface transfer data directly to or from the memory, without continuous involvement by the processor.

1.2 INTERRUPTS

There are many situations where other tasks can be performed while waiting for an I/O device to become ready. To allow this to happen, we can arrange for the I/O device to alert the processor when it becomes ready. It can do so by sending a hardware signal called an interrupt to the processor.

At least one of the bus control lines, called the interrupt request line, is usually dedicated for interrupt. Since the processor is no longer required to continuously check the status of external devices, it can use the waiting period to perform other useful functions. Indeed, by using interrupts, such waiting periods can ideally be eliminated.

Example to illustrate Interrupt Operation

Simple approach:

- Consider a task that requires some computations to be performed and the results to be printed on a line printer. This is followed by more computations and output, and so on.
- Let the program consist of two routines, **COMPUTE** and **PRINT**. Assume that **COMPUTE** produces a set of n lines of output, to be printed by the **PRINT** routine.
- The required task may be performed by repeatedly executing first the **COMPUTE** routine and then the **PRINT** routine.
- The printer accepts only one line of text at a time. Hence, the **PRINT** routine must send one line of text, wait for it to be printed, and then send the next line, and so on, until all the results have been printed.
- The disadvantage of this simple approach is that the processor spends a considerable amount of time waiting for the printer to become ready.

Efficient approach:

- If it is possible to overlap printing and computation, that is, to execute the **COMPUTE** routine while printing is in progress, a faster overall speed of execution will result. This may be achieved as follows.
- First, the **COMPUTE** routine is executed to produce the first n lines of output.
- Then, the **PRINT** routine is executed to send the first line of text to the printer. At this point, instead of waiting for the line to be printed, the **PRINT** routine may be temporarily suspended and execution of the **COMPUTE** routine continued.
- Whenever the printer becomes ready, it alerts the processor by sending an interrupt request signal. In response, the processor interrupts execution of the **COMPUTE** routine and transfers control to the **PRINT** routine.
- The **PRINT** routine sends the second line to the printer and is again suspended. Then the interrupted **COMPUTE** routine resumes execution at the point of interruption. This process continues until all n lines have been printed and the **PRINT** routine ends.
- The **PRINT** routine will be restarted whenever the next set of m lines is available for printing.
- If **COMPUTE** takes longer to generate n lines than the time required to print them, the

processor will be performing useful computations all the time.

This example illustrates the concepts of **interrupts**. The routine executed in response to an interrupt request is called the **interrupt service routine[ISR]**.

Interrupt resemblance to Subroutines

- Assume that an interrupt request arrives during execution of instruction i in Figure 1. 5.
- The processor first completes execution of instruction i . Then, it loads the program counter with the address of the first instruction of the interrupt-service routine.
- After execution of the interrupt-service routine, the processor has to come back to instruction $i+1$.
- Therefore, when an interrupt occurs, the current contents of the PC, which point to instruction $i+1$ must be put in temporary storage in a known location.
- A Return-from-interrupt instruction at the end of the interrupt-service routine reloads the PC from that temporary storage location, causing execution to resume at instruction $i+1$.

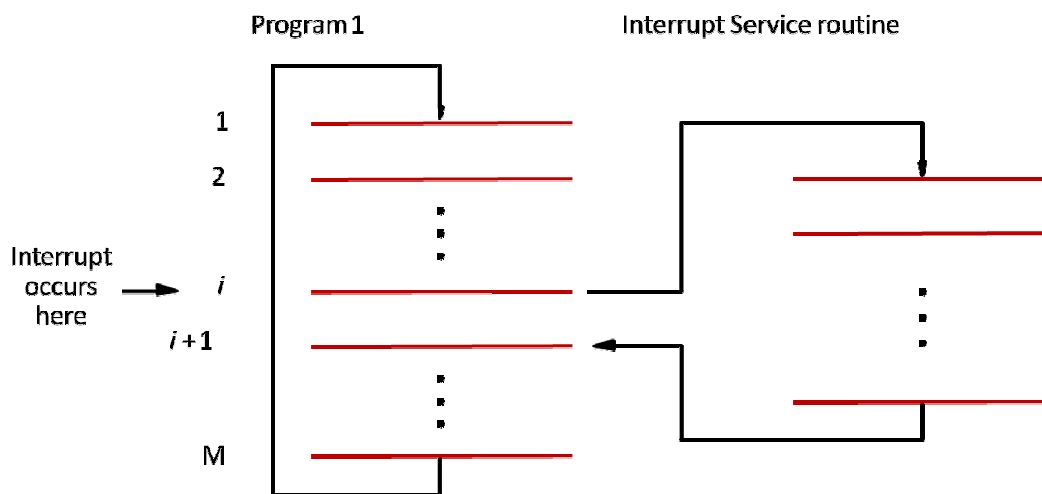


Figure 1.5 Transfer of control through the use of interrupts

- In many processors, the return address is saved on the processor stack. Alternatively, it may be saved in a special location, such as a register provided for this purpose.
- We should note that as part of handling interrupts, the processor must inform the device that its request has been recognized so that it may remove its interrupts –request signal. This may be accomplished by means of a special control signal on the bus.

Difference between Interrupt and Subroutines

- So far, treatment of an interrupt-service routine is very similar to that of a subroutine. An important departure from this similarity should be noted.
- A subroutine performs a function required by the program from which it is called.
- However, the interrupt- service routine may not have anything in common with the program being executed at the time the interrupt request is received. In fact, the two programs often belong to different users.
- Therefore, before starting execution of the interrupt-service routine, any information that may be altered during the execution of that routine must be saved. This information must be restored before execution of the interrupted program is resumed.
- In this way, the original program can continue execution without being affected in any way by the interruption, except for the time delay.

Saving and restoring information when Interrupt occurs:

- The task of saving and restoring information can be done automatically by the processor or by program instructions.
- Most modern processors save only the minimum amount of information needed to maintain the integrity of program execution.
- This is because the process of saving and restoring registers involves memory transfers that increase the total execution time, and hence represent execution overhead.
- Saving registers also increases the delay between the time an interrupt request is received and the start of execution of the interrupt-service routine. This delay is called interrupt latency.
- Typically, the processor saves only the contents of the program counter[PC] and the Processor Status Word register[PSW].
- Any additional information that needs to be saved must be saved by program instructions at the beginning of the interrupt-service routine and restored at the end of the routine.

Summary

- An interrupt is more than a simple mechanism for coordinating I/O transfers.
- In a general sense, interrupts enable transfer of control from one program to another to be

initiated by an event external to the computer.

- Execution of the interrupted program resumes after the execution of the interrupt-service routine has been completed.
- The concept of interrupts is used in operating systems and in many control applications where processing of certain routines must be accurately timed relative to external events. The latter type of application is referred to as real-time processing.

1.2.1 INTERRUPT HARDWARE

- I/O device requests an interrupt by activating a bus line called interrupt-request.
- Most computers are likely to have several I/O devices that can request an interrupt.
- A single interrupt-request line may be used to serve n devices as depicted in Figure 1.6.

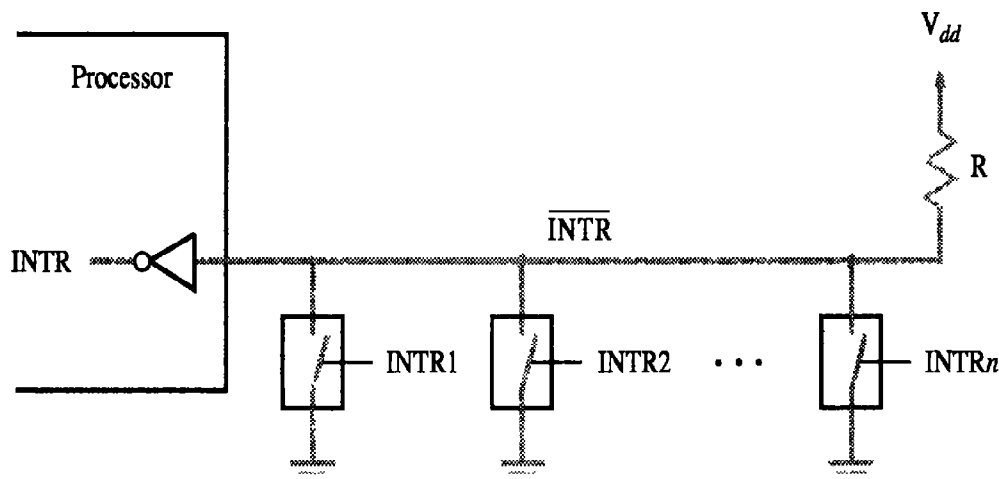


Figure 1.6 An equivalent circuit for an open-drain bus used to implement a common request line

- All devices are connected to the line via switches to ground. To request an interrupt, a device closes its associated switch.
- Thus, if all interrupt-request signals $INTR_1$, to $INTR_n$, 1 are inactive, that is, if all switches are open, the voltage on the interrupt-request line will be equal to V_{dd} . This is the inactive state of the line.
- When a device requests an interrupt by closing its switch, the voltage on the line drops to 0, causing the interrupt-request signal, $\overline{INTR}$, received by the processor to go to 1.
- Since the closing of one or more switches will cause the line voltage to drop to 0, the value of $\overline{INTR}$ is the logical OR of the requests from individual devices, that is,

$$\text{INTR} = \text{INTR}_1 + \dots + \text{INTR}_n$$

- It is customary to use the complemented form, $\overline{\text{INTR}}$, to name the interrupt-request signal on the common line, because this signal is active when in the low-voltage state.

Electronic Implementation of Interrupt Circuit

- In the electronic implementation of the circuit in Figure 1. 6, special gates known as **open-collector** (for bipolar circuits) or **open-drain** (for MOS circuits) are used to drive the **INTR** line.
- The output of an *open-collector* or an *open-drain* gate is equivalent to a switch to ground that is open when the gate's input is in the 0 state and closed when it is in the 1 state.
- The voltage level, hence the logic state, at the output of the gate is determined by the data applied to all the gates connected to the bus, according to the equation given above.
- Resistor **R** is called a pull-up resistor because it pulls the line voltage up to the high-voltage state when the switches are open.

1.2.2 ENABLING AND DISABLING INTERRUPT

- The arrival of an interrupt request from an external device causes the processor to suspend the execution of one program and start the execution of another.
- Because interrupts can arrive at any time, they may alter the sequence of events from that envisaged by the programmer. Hence, the interruption of program execution must be carefully controlled.
- A fundamental facility found in all computers is the ability to enable and disable such interruptions as desired.

Need for enabling and disabling interrupt

- There are many situations in which the processor should ignore interrupt requests.

Case 1:

- ❖ In case of Print program an interrupt request from the printer should be accepted only if there are output lines to be printed. After printing the last line of a set of n lines, interrupts should be disabled until another set becomes available for printing.

Case 2:

- ❖ In another case, it may be necessary to guarantee that a particular sequence of

instructions is executed to the end without interruption because the interrupt-service routine may change some of the data used by the instructions in question.

- For these reasons, some means for enabling and disabling interrupts must be available to the programmer.
- A simple way is to provide machine instructions, such as Interrupt-enable and Interrupt-disable, that perform these functions.

Single Interrupt Request Situation

- When a device activates the interrupt-request signal, it keeps this signal activated until it learns that the processor has accepted its request.
- This means that the interrupt-request signal will be active during execution of the interrupt-service routine, perhaps until an instruction is reached that accesses the device in question,
- It is essential to ensure that this active request signal does not lead to successive interruptions, causing the system to enter an infinite loop from which it cannot recover.
- Several mechanisms are available to solve this problem.
- We will describe three possibilities here;

First Possibility

- ❖ The first possibility is to have the processor hardware ignore the interrupt-request line until the execution of the first instruction of the interrupt-service routine has been completed.
- ❖ Then, by using an Interrupt-disable instruction as the first instruction in the interrupt-service routine, the programmer can ensure that no further interruptions will occur until an Interrupt-enable instruction is executed.
- ❖ Typically, the Interrupt-enable instruction will be the last instruction in the interrupt-service routine before the Return-from-interrupt instruction.
- ❖ The processor must guarantee that execution of the Return-from-interrupt instruction is completed before further interruption can occur.

Second Possibility

- ❖ The second option, which is suitable for a simple processor with only one interrupt-

request line, is to have the processor automatically disable interrupts before starting the execution of the interrupt-service routine.

- ❖ After saving the contents of the PC and the **processor status register (PS)** on the stack, the processor performs the equivalent of executing an Interrupt-disable instruction.
- ❖ It is often the case that one bit in the PS register, called Interrupt-enable, indicates whether interrupts are enabled. An interrupt request received while this bit is equal to 1 will be accepted.
- ❖ After saving the contents of the PS on the stack, with the Interrupt-enable bit equal to 1, the processor clears the Interrupt-enable bit in its PS register, thus disabling further interrupts.
- ❖ When a Return-from-interrupt instruction is executed, the contents of the PS are restored from the stack, setting the Interrupt-enable bit back to 1. Hence, interrupts are again enabled.

Third Possibility

- ❖ In the third option, the processor has a special interrupt-request line for which the interrupt-handling circuit responds only to the leading edge of the signal.
- ❖ Such a line is said to be edge-triggered. In this case, the processor will receive only one request, regardless of how long the line is activated.
- ❖ Hence, there is no danger of multiple interruptions and no need to explicitly disable interrupt requests from this line.

Summary of Interrupt Sequence for Single Device

Assuming that interrupts are enabled, the following is a typical scenario

1. The device raises an interrupt request.
2. The processor interrupts the program currently being executed.
3. Interrupts are disabled by changing the control bits in the PS (except in the case of edge-triggered interrupts).
4. The device is informed that its request has been recognized, and in response, it deactivates the interrupt-request signal.
5. The action requested by the interrupt is performed by the interrupt-service routine.

6. Interrupts are enabled and execution of the interrupt program is resumed.

1.2.3. HANDLING MULTIPLE DEVICES

- Let us now consider the situation where a number of devices capable of initiating interrupts are connected to the processor.
- Because these devices are operationally independent. There is no definite order in which they will generate interrupts.
- For example, device X may request an interrupt while an interrupt caused by device Y is being serviced, or several devices may request interrupts at exactly the same time. This gives rise to a number of questions

1. How can the processor recognize the device requesting an interrupt?
2. Given that different devices are likely to require different interrupt-service routines, how can the processor obtain the starting address of the appropriate routine in each case?
3. Should a device be allowed to interrupt the processor while another interrupt is being serviced?
4. How should two or more simultaneous interrupt requests be handled?

- The means by which these problems are resolved vary from one computer to another, and the approach taken is an important consideration in determining the computer's suitability for a given application.
- If two devices have activated the line at the same time, it must be possible to break the tie and select one of the two requests for service.
- When the interrupt-service routine for the selected device has been completed, the second request can be serviced.
- The information needed to determine whether a device is requesting an interrupt is available in its status register.
- When a device raises an interrupt request, it sets to 1 one of the bits in its status register, which we will call the IRQ bit.
- The simplest way to identify the interrupting device is to have the interrupt-service routine poll all the I/O devices connected to the bus.
- The first device encountered with its IRQ bit set is the device that should be serviced.
- An appropriate subroutine is called to provide the requested service.

Vectored Interrupt

The polling scheme is easy to implement. Its main disadvantage is the time spent interrogating the IRQ bits of all the devices that may not be requesting any service. An alternative approach is to use vectored interrupts.

- To reduce the time involved in the polling process, a device requesting an interrupt may identify itself directly to the processor. Then, the processor can immediately start executing the corresponding interrupt-service routine. The term vectored interrupts refers to all interrupt-handling schemes based on this approach.
- A device requesting an interrupt can identify itself by sending a special code to the processor over-the bus.
- This enables the processor to identify individual devices even if they share a single interrupt-request line. The code supplied by the device may represent the starting address of the interrupt-service routine for that device.
- The code length is typically in the range of 4 to 8 bits.
- The remainder of the address is supplied by the processor based on the area in its memory where the addresses for interrupt-service routines are located.
- This arrangement implies that the interrupt-service routine for a given device must always start at the same location. The programmer can gain some flexibility by storing in this location an instruction that causes a branch to the appropriate routine.
- In many computers, this is done automatically by the interrupt-handling mechanism.
- The location pointed to by the interrupting device is used to store the starting address of the interrupt-service routine.
- The processor reads this address, called the interrupt vector and loads it into the PC.
- The interrupt vector may also include a new value for the processor status register.

Interrupt Request over Data Bus

- In most computers, I/O devices send the interrupt-vector code over the data bus, using the bus control signals to ensure that devices do not interfere with each other.
- When a device sends an interrupt request, the processor may not be ready to receive the interrupt-vector code immediately.
- For example, it must first complete the execution of the current instruction, which may

require the use of the bus.

- There may be further delays if interrupts happen to be disabled at the time the request is raised.
- The interrupting device must wait to put data on the bus only when the processor is ready to receive it.
- When the processor is ready to receive the interrupt-vector code, it activates the interrupt-acknowledge line, INTA. The I/O device responds by sending its interrupt-vector code and turning off the INTR signal.

Interrupt Nesting

- Interrupts should be disabled during the execution of an interrupt-service routine, to ensure that a request from one device will not cause more than one interruption.
- The same arrangement is often used when several devices are involved, in which case execution of a given interrupt-service routine, once started, always continues to completion before the processor accepts an interrupt request from a second device.
- Interrupt-service routines are typically short, and the delay they may cause is acceptable for most simple devices.
- For some devices, however, a long delay in responding to an interrupt request may lead to erroneous operation.

Example: A computer that keeps track of the time of day using a real-time clock.

- This is a device that sends interrupt requests to the processor at regular intervals.
- For each of these requests, the processor executes a short interrupt-service routine to increment a set of counters in the memory that keep track of time in seconds, minutes, and so on.
- Proper operation requires that the delay in responding to an interrupt request from the real-time clock be small in comparison with the interval between two successive requests.
- To ensure that this requirement is satisfied in the presence of other interrupting devices, it may be necessary to accept an interrupt request from the clock during the execution of an interrupt-service routine for another device.

This example suggests that I/O devices should be organized in a priority structure. An interrupt request from a high-priority device should be accepted while the

processor is servicing another request from a lower-priority device.

- A multiple-level priority organization means that during execution of an interrupt-service routine, interrupt requests will be accepted from some devices but not from others, depending upon the device's priority.
- To implement this scheme, we can assign a priority level to the processor that can be changed under program control.
- The priority level of the processor is the priority of the program that is currently being executed.
- The processor accepts interrupts only from devices that have priorities higher than its own.
- At the time the execution of an interrupt-service routine for some device is started, the priority of the processor is raised to that of the device. This action disables interrupts from devices at the same level of priority or lower.
- However, interrupt requests from higher-priority devices will continue to be accepted.

Privileged Mode

- The processor's priority is usually encoded in a few bits of the processor status word. It can be changed by program instructions that write into the PS. These are privileged instructions, which can be executed only while the processor is running in the supervisor mode.
- The processor is in the supervisor mode only when executing operating system routines.
- It switches to the user mode before beginning to execute application programs. Thus, a user program cannot accidentally, or intentionally, change the priority of the processor and disrupt the system's operation.
- An attempt to execute a privileged instruction while in the user mode leads to a special type of interrupt called a privilege exception.

Multiple-Priority scheme can be implemented easily by using separate interrupt-request and interrupt-acknowledge lines for each device, as shown in Figure 1. 7.

- Each of the interrupt-request lines is assigned a different priority level. Interrupt requests received over these lines are sent to a priority arbitration circuit in the processor.
- A request is accepted only if it has a higher priority level than that currently assigned to the processor.

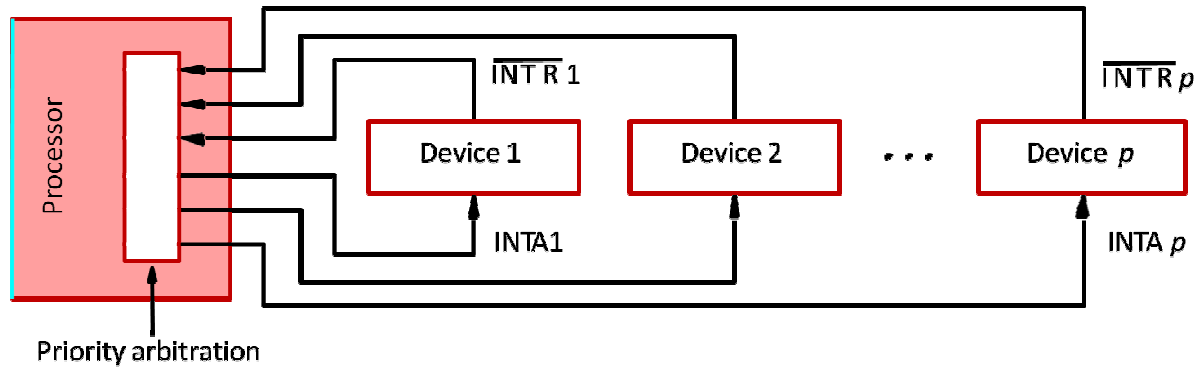


Figure 1.7 Implementation of interrupt priority using individual interrupt-request and acknowledge lines

Simultaneous Requests

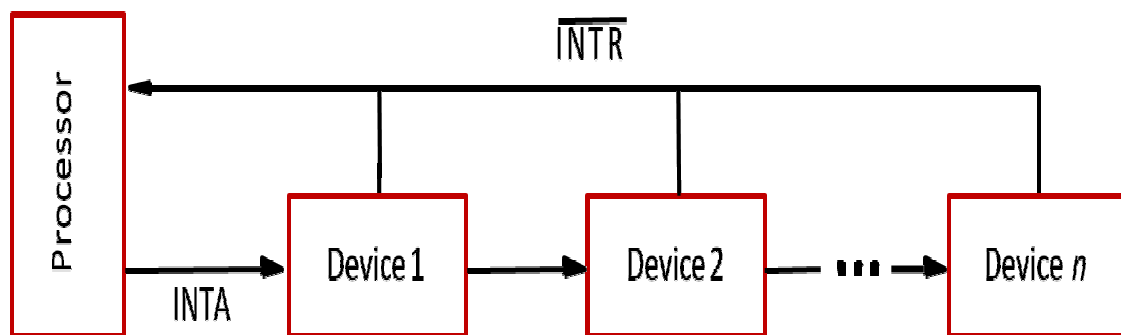
- Let us now consider the problem of simultaneous arrivals of interrupt requests from two or more devices.
- The processor must have some means of deciding which request to service first.

Polling:

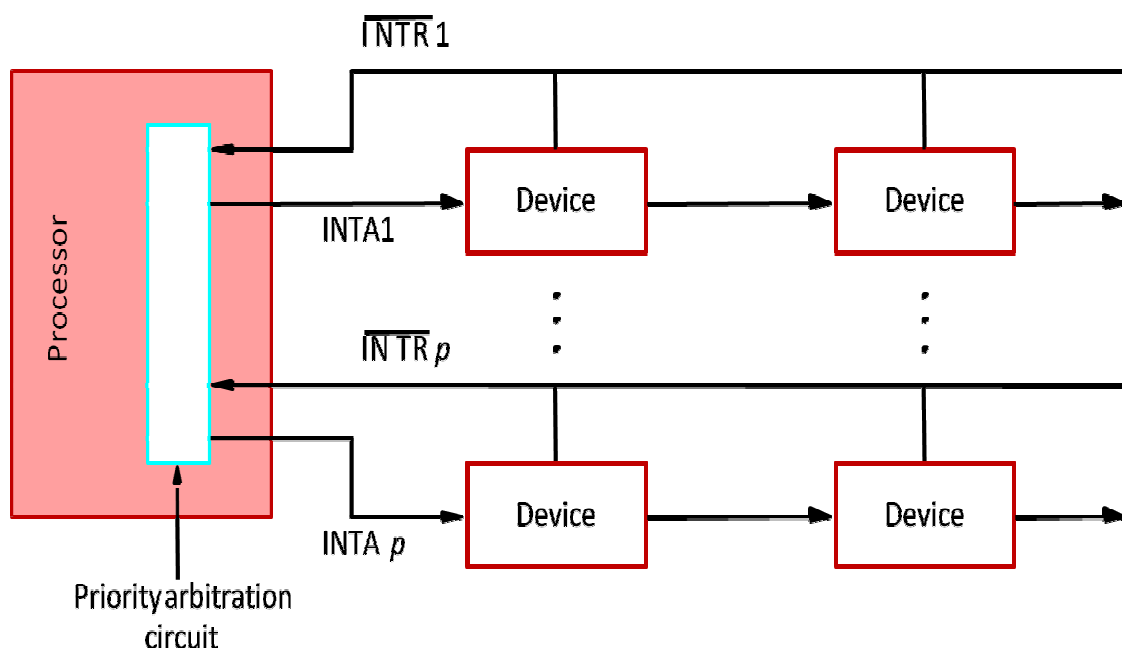
- Polling the status registers of the I/O devices is the simplest mechanism.
- In this case priority is determined by the order in which the devices are polled. When vectored interrupts are used, we must ensure that only one device is selected to send its interrupt vector code.
- A widely used scheme is to connect the devices to form a Daisy Chain, as shown in Figure 1. 8a.
- The interrupt-request line INTR is common to all devices. The interrupt-acknowledge line, INTA, is connected in a daisy-chain fashion, such that the INTA signal propagates serially through the devices.
- When several devices raise an interrupt request and the INTR line is activated, the processor responds by setting the INTA line to 1.
- This signal is received by device 1. Device passes the signal on to device 2 only if it does not require any service.
- If device 1 has a pending request for interrupt, it blocks the INTA signal and proceeds to put its identifying code on the data lines.
- Therefore, in the daisy-chain arrangement, the device that is electrically closest to the processor has the highest priority. The second device along the chain has second highest

priority, and so on.

- The scheme in Figure 4. 8a requires considerably fewer wires than the individual connections in Figure 4. 7.
- The main advantage of the scheme in Figure 4. 7 is that it allows the processor to accept interrupt requests from some devices but not from others, depending upon their priorities.
- The two schemes may be combined to produce the more general structure in Figure 4.8b.
- Devices are organized in groups, and each group is connected at a different priority level. Within a group, devices are connected in a daisy chain. This organization is used in many computer systems.



(a) Daisy Chain



(b) Arrangement of priority groups

Figure 1.8 Interrupt Priority Schemes

1.2.3 CONTROLLING DEVICE REQUESTS

- Until now, we have assumed that an I/O device interface generates an interrupt request whenever it is ready for an I/O transfer.
- It is important to ensure that interrupt requests are generated only by those I/O devices that are being used by a given program.
- Idle devices must not be allowed to generate interrupt requests, even though they may be ready to participate in I/O transfer operations.
- Hence, we need a mechanism in the interface circuits of individual devices to control whether a device is allowed to generate an interrupt request.
- The control needed is usually provided in the form of an interrupt-enable bit in the device's interface circuit.
- There are two independent mechanisms for controlling interrupt requests.
- At the device end, an interrupt-enable bit in a control register determines whether the device is allowed to generate an interrupt request.
- At the processor end, either an interrupt enable bit in the **PS** register or a priority structure determines whether a given interrupt request will be accepted.

Example for Controlling Device Requests

- Consider a processor that uses the vectored interrupt scheme, where the starting address of the interrupt-service routine is stored at memory location **INTVEC**.
- Interrupts are enabled by setting to an interrupt-enable bit, **IE**, in the processor status word, which we assume is bit 9.
- A keyboard and a display unit connected to this processor have the status, control, and data registers shown in Figure 1. 3.
- Assume that at some point in a program called **Main** we wish to read an input line from the keyboard and store the characters in successive byte locations in the memory, starting at location **LINE**.
- To perform this operation using interrupts, we need to initialize the interrupt process. This may be accomplished as follows :
 1. Load the starting address of the interrupt-service routine in location **INTVEC**.
 2. Load the address **LINE** in a memory location **PNTR**. The interrupt-service routine will use this location as a pointer to store the input characters in the memory.

3. Enable keyboard interrupts by setting bit 2 in register **CONTROL** to 1.
4. Enable interrupts in the processor by setting to 1 the **IE** bit in the processor status register, **PS**.

- Once this initialization is completed, typing a character on the keyboard will cause an interrupt request to be generated by the keyboard interface.
- The program being executed at that time will be interrupted and the interrupt-service routine will be executed. This routine has to perform the following tasks:

1. Read the input character from the keyboard input data register. This will cause the interface circuit to remove its interrupt request.
2. Store the character in the memory location pointed to by **PNTR**, and increment **PNTR**.
3. When the end of the line is reached, disable keyboard interrupts and inform program **Main**.
4. Return from interrupt.

- The instructions needed to perform these tasks are shown in Figure 1.9.
- When, the end of the input line is detected, the interrupt-service routine clears the **KEN** bit in register **CONTROL** as no further input is expected.
- It also sets to 1 the variable **EOL** (End of Line). This variable is initially set to **0**.
- We assume that it is checked periodically by program **Main** to determine when the input line is ready for processing.

Main Program

Move	#LINE,PNTR	Initialize buffer pointer.
Clear	EOL	Clear end-of-line indicator.
BitSet	#2,CONTROL	Enable keyboard interrupts.
BitSet	#9,PS	Set interrupt-enable bit in the PS.
.		
.		
.		

Interrupt-service routine

READ	Move Multiple	R0-R1,-(SP)	Save registers R0 and R1 on stack.
	Move	PNTR,R0	Load address pointer.
	MoveByte	DATAIN,R1	Get input character and
	MoveByte	R1,(R0)+	store it in memory,
	Move	R0,PNTR	Update pointer.
	CompareByte	#\$0D,R1	Check if Carriage Return.
	Branch≠0	RTRN	
	Move	#1,EOL	Indicate end of line.
	BitClear	#2,CONTROL	Disable keyboard interrupts.
RTRN	MoveMultiple	(SP)+,R0-R1	Restore registers R0 and R1.
	Return-from-interrupt		

Figure 1.9 Using interrupts to read a line of characters from a keyboard via the registers in Figure 1.3

1. 4 DIRECT MEMORY ACCESS [DMA]

Why DMA?

- When interrupts are used, there is the additional overhead associated with saving and restoring the program counter and other state information.
- To transfer large blocks of data at high speed, an alternative approach is used. A special control unit may be provided to allow transfer of a block of data directly between an external device and the main memory, without continuous intervention by the processor. This approach is called **Direct Memory Access** or **DMA**.

DMA Controller

- DMA transfers are performed by a control circuit that is part of the I/O device interface. We refer to this circuit as a DMA Controller.
- The DMA controller performs the functions that would normally be carried out by the processor when accessing the main memory.

- For each word transferred, it provides the memory address and all the bus signals that control data transfer.
- Since it has to transfer blocks of data, the DMA controller must increment the memory address for successive words and keep track of the number of transfers.
- Although a DMA controller can transfer data without intervention by the processor, its operation must be under the control of a program executed by the processor.
- To initiate the transfer of a block of words, the processor sends the starting address, the number of words in the block, and the direction of the transfer. On receiving this information, the DMA controller proceeds to perform the requested operation.
- When the entire block has been transferred, the controller informs the processor by raising an interrupt signal.
- While a DMA transfer is taking place, the program that requested the transfer cannot continue, and the processor can be used to execute another program.
- After the DMA transfer is completed, the processor can return to the program that requested the transfer.

I/O operations

- I/O operations are always performed by the operating system of the computer in response to a request from an application program.
- The OS is also responsible for suspending the execution of one program and starting another.
- Thus, for an I/O operation involving DMA, the OS puts the program that requested the transfer in the Blocked state, initiates the DMA operation, and starts the execution of another program.
- When the transfer is completed, the DMA controller informs the processor by sending an interrupt request.
- In response, the OS puts the suspended program in the Runnable state so that it can be selected by the scheduler to continue execution.

DMA Controller Register

- Figure 1. 10 shows an example of the DMA controller registers that are accessed by the processor to initiate transfer operations.

- Two registers are used for storing the starting address and the word count.
- The third register contains status and control flags.
- The R/W bit determines the direction of the transfer. When this bit is set to 1 by a program instruction, the controller performs a read operation, that is, it transfers data from the memory to the I/O device. Otherwise, it performs a write operation.
- When the controller has completed transferring a block of data and is ready to receive another command, it sets the **Done** flag to 1.
- Bit 30 is the Interrupt-enable flag, **IE**. When this flag is set to 1, it causes the controller to raise an interrupt after it has completed transferring a block of data.
- Finally, the controller-sets the IRQ bit to 1 when it has requested an interrupt.

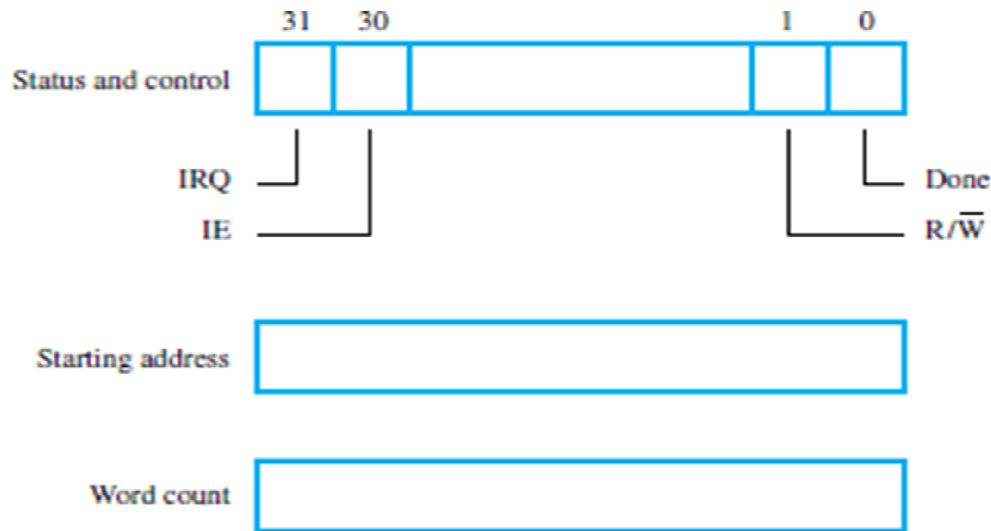


Figure 1.10 Registers in a DMA interface.

- An example of a computer system is given in Figure 1. 11, showing how DMA controllers may be used.
- A DMA controller connects a high-speed network to the computer bus.
- The disk controller, which controls two disks, also has DMA capability and provides two DMA channels. It can perform two independent DMA operations, as if each disk had its own DMA controller.
- The registers needed to store the memory address, the word count, and so on are duplicated, so that one set can be used with each device.

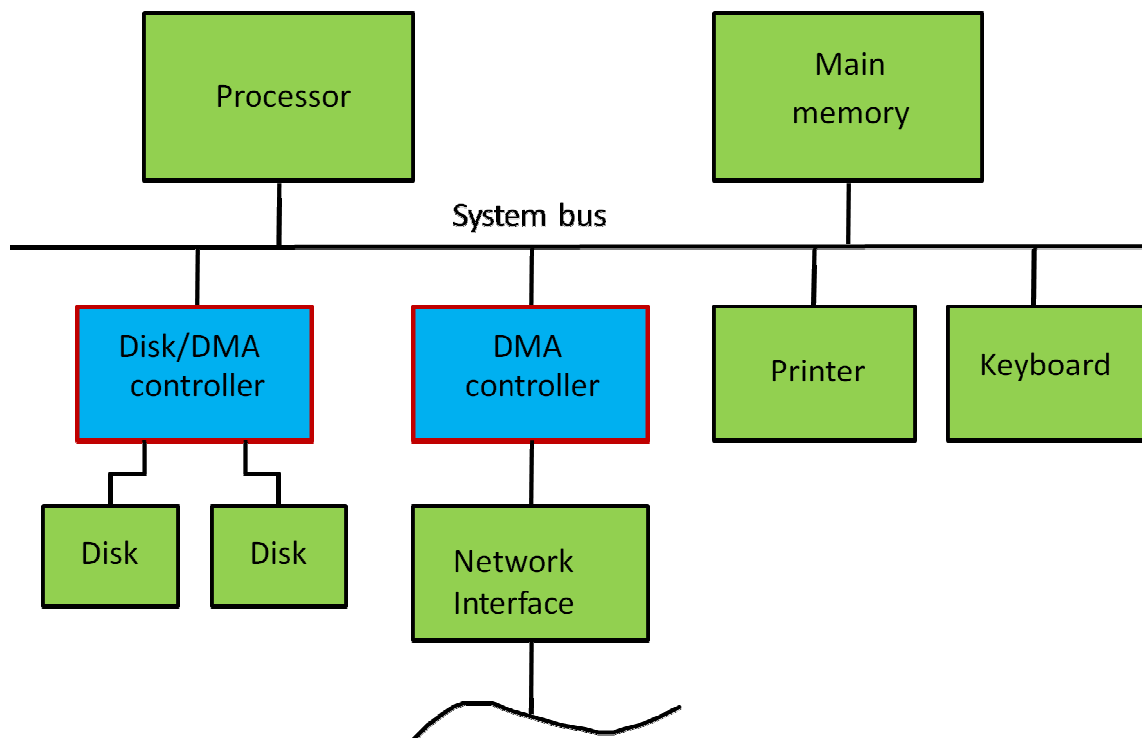


Figure 1.11 Registers in a DMA interface.

DMA data transfer between main memory and disk

- To start a DMA transfer of a block of data from the main memory to one of the disks, a program writes the address and word count information into the registers of the corresponding channel of the disk controller.
- It also provides the disk controller with information to identify the data for future retrieval.
- The DMA controller proceeds independently to implement the specified operation.
- When the DMA transfer is completed, this fact is recorded in the status and control register of the DMA channel by setting the Done bit. At the same time, if the IE bit is set, the controller sends an interrupt request to the processor and sets the IRQ bit.
- The status register can also be used to record other information, such as whether the transfer took place correctly or errors occurred.
- Memory accesses by the processor and the DMA controllers are interwoven. Requests by DMA devices for using the bus are always given higher priority than processor requests.
- Among different DMA devices, top priority is given to high-speed peripherals such as a disk, a high-speed network interface, or a graphics display device.

- Since the processor originates most memory access cycles, the DMA controller can be said to “***steal***” memory cycles from the processor.
- Hence, this interweaving technique is usually called ***cycle stealing***.
- Alternatively, the DMA controller may be given exclusive access to the main memory to transfer a block of data without interruption. This is known as **block** or **burst** mode.
- Most DMA controllers incorporate a data storage buffer. In the case of the network interface in Figure 1. 11, for example, the DMA controller reads a block of data from the main memory and stores it into its input buffer.
- This transfer takes place using burst mode at a speed appropriate to the memory and the computer bus.
- Then, the data in the buffer are transmitted over the network at the speed of the network.
- A conflict may arise if both the processor and a DMA controller or two DMA controllers try to use the bus at the same time to access the main memory.
- To resolve these conflicts, an arbitration procedure is implemented on the bus to coordinate the activities of all devices requesting memory transfers.

PART 2**QUESTION BANK**

1. Write a brief note on accessing I/O devices
2. Explain Program-Controlled I/O with example
3. What is an interrupt? Illustrate interrupt operation with example
4. Compare and contrast Interrupt with Subroutine
5. Write a note on interrupt hardware
6. Write a note on enabling and disabling of interrupt.
7. Explain three methods to solve interrupt request from single device
8. Explain vectored interrupt
9. Explain interrupt nesting
10. Explain how simultaneous interrupt requests are handled by processor
11. With example explain how device request are controlled
12. What is DMA?
13. Explain how DMA controller controls DMA transfers
14. With neat diagram explain DMA controller register

PART 3**EXERCISE PROBLEMS AND SOLUTION**

Q 1. The input status bit in an interface circuit is cleared as soon as the input data buffer is read. Why is this important?

SOLUTION:

After reading the input data, it is necessary to clear the input status flag before the program begins a new read operation. Otherwise, the same input data would be read a second time.

Q 2. Write a program that displays the contents of **10** bytes of the main memory in hexadecimal format on a video display. Use either the assembler instructions of a processor of your choice or pseudo-instructions. Start at location **LOC** in the memory, and use two hex characters per byte. The contents of successive bytes should be separated by a space.

SOLUTION:

The ASCII code for the numbers 0 to 9 can be obtained by adding \$30 to the number. The values 10 to 15 are represented by the letters A to F, whose ASCII codes can be obtained by adding \$37 to the corresponding binary number.

Assume the output status bit is **b₄** in register Status, and the output data register is Output.

Move	#10,R0	Use R0-as counter
Move	#LOC,RI	Use R1 as pointer
Move	(R1),R2	Get next byte
Move	R2,R3	
Shift-right	#4, R3	Prepare bits b ₇ -b ₄
Call	Convert	
Move	R2,R3	Prepare bits b ₃ - b ₀
Call	Convert	
Move	\$20,R3	Print space
Call	Print	

	Increment	R1	
	Decrement	R0	
	Branch > 0	Next	Repeat if more bytes left
	End		
Convert	And	#0F,R3	Keep only low-order 4 bits
	Compare	#9,R3	
	Branch ≥ 0	Letters	Branch if [R3] ≥ 9
	Or	#\$30,R3	Convert to ASCII, for values 0 to 9
	Branch	Print	
Letters	Add	#\$37,R3	Convert to ASCII, for values 10 to 15
Print	BitTest	#4,Status	Test output status bit
	Branch=0	Print	Loop back if equal to 0
	Move	R3,Output	Send character to output register
	Return		

Q 3. The address bus of a computer has 16 address lines, A15-o. If the address assigned to one device is 7CA416 and the address decoder for that device ignores lines A8 and A9, what are all the addresses to which this device will respond?

SOLUTION:

7CA4, 7DA4, 7EA4, 7FA4.

Q 4.What is the difference between a subroutine and an interrupt-service routine?

SOLUTION:

- ✓ A subroutine is called by a program instruction to perform a function needed by the calling program.
- ✓ An interrupt-service routine is initiated by an event such as an input operation or a hardware error. The function it performs may not be at all related to the program being executed at the time of interruption. Hence, it must not affect any of the data or status information relating to that program.

Q 5. The discussion in this chapter assumed that interrupts are not acknowledged until the current machine instruction completes execution. Consider the possibility of suspending operation of the processor in the middle of executing an instruction in order to acknowledge an interrupt. Discuss the difficulties that may arise.

SOLUTION:

- ✓ If execution of the interrupted instruction is to be completed after return from interrupt, a large amount of information needs to be saved. This includes the contents of any temporary registers, intermediate results, etc.
- ✓ An alternative is to abort the interrupted instruction and start its execution from the beginning after return from interrupt. In this case, the results of an instruction must not be stored in registers or memory locations until it is guaranteed that execution of the instruction will be completed without interruption.